DATASTRAW TECH INTERNSHIP ASSESSMENT TEST

Hiring Assignment: Build a Smart Roster Management System

Submission deadline: 4 days | Expected effort: Approximately 1.5-2 days of focused work

---

1. The Problem

Imagine you are working with a growing customer experience and operations company that manages support and operational processes for multiple D2C brands.

The company has a team of at least 30 employees distributed across multiple brands, processes, and operational pods. These processes may include Customer Support (CS), Abandoned Cart Conversions (ACC), retention, Non-Delivery Report management (NDR), and other brand-specific operations.

The team structure is not uniform.

- Approximately 80% of the brands are managed by a single dedicated employee who handles all the processes assigned to that brand.
- The remaining brands typically have two or three employees assigned to them.
- When multiple employees work on the same brand, they may handle different processes. For example, one employee may manage Customer Support, another may handle Abandoned Cart Conversions, and a third may handle NDR.
- In some cases, multiple employees may work on the same process for a brand.
- Employees are grouped into operational pods, each managed by a pod lead.

Every brand has its own operational requirements. Some processes need coverage 365 days a year, including Sundays and public holidays. Others operate every day except public holidays, while some have Sundays and public holidays off.

Every month, the operations team needs to prepare a roster for all employees while ensuring that client-specific coverage requirements are met.

The roster must account for weekly offs, public holidays, planned leaves, comp-offs, employee availability, process-specific working rules, and resource allocation across brands.

As the team grows, manually planning and validating these schedules becomes increasingly complex. A weekly off assigned to an employee could leave an entire brand without coverage, while a change in one person's availability could affect several client accounts.

Your task is to build a working roster management application that helps an operations manager plan, generate, validate, and manage monthly rosters across multiple brands, processes, and pods.

The objective is to solve the underlying scheduling problem, not simply create a calendar or digitise an existing spreadsheet.

2. What You'll Build

Build a browser-based roster management tool that allows an operations team to:

- Maintain a central database of employees, brands, processes, pods, pod leads, and resource allocations.
- Define working-day rules and minimum coverage requirements for each brand and process.
- Configure employee availability, weekly offs, planned leaves, comp-offs, and public holidays.
- Generate or assist in generating monthly rosters for the entire team.
- View schedules at the company, pod, brand, process, and individual employee levels.
- Identify scheduling conflicts and coverage gaps before publishing a roster.
- Make manual changes and validate the impact of those changes.
- Handle changes to employee availability or brand requirements without having to rebuild the entire roster manually.

The tool should be designed to support at least 30 employees across multiple brands and processes, with the flexibility to accommodate additional employees, brands, and scheduling rules.

3. Core Requirements

A. Team, Brand and Process Management

The tool should support:

- At least 30 employees.
- Multiple brands and operational pods.
- Multiple processes, with different processes assigned to different employees.
- One or more employees assigned to each brand.
- Pod leads responsible for overseeing their respective teams.
- Employees assigned to multiple brands or processes, where applicable.

The sample dataset should reflect realistic staffing patterns:

- Approximately 80% of brands should have one dedicated employee handling all processes assigned to that brand.
- The remaining brands should generally have two or three employees.
- Where a brand has multiple employees, they should usually be assigned to different processes, although overlapping process assignments should also be represented.
- Some processes may require more than one employee to maintain sufficient coverage.

The system should allow an operations manager to add, edit, or remove assignments without requiring changes to the application's underlying code.

B. Client-Specific Working Rules

Each brand or process may have different operating requirements.

The system should support configurable rules such as:

- 365-day operations, including Sundays and public holidays.
- Daily operations with public holidays off.
- Sundays and public holidays off.
- Different coverage requirements for different processes within the same brand.
- Minimum staffing requirements on specific days.
- Different weekly-off patterns for employees assigned to the same brand.

For example, a brand may have Customer Support operating seven days a week while its other process follows a different schedule.

The system should distinguish between a brand's operating calendar and the working schedule of an individual employee.

C. Monthly Roster Generation

The operations manager should be able to select a month and generate or prepare a roster for the entire team.

The scheduling logic should consider:

- Brand and process operating calendars.
- Employee assignments and availability.
- Weekly offs and applicable public holidays.
- Planned leaves and approved comp-offs.
- Minimum staffing and coverage requirements.
- Employees working across multiple brands or processes.
- Employee preferences, where applicable.

The roster should show who is working, who is off, and whether each brand and process has adequate coverage on each required working day.

The application should not assume that every employee follows the same schedule or that every brand operates on the same days.

If the scheduling requirements cannot all be satisfied, the application should identify the conflict and explain what needs attention rather than silently generating an invalid roster.

D. Conflict Detection and Validation

Before a roster is finalised, the system should identify issues such as:

- A brand or process requiring coverage when no assigned employee is available.
- An employee being assigned to overlapping responsibilities or incompatible shifts.
- Weekly offs or leave conflicting with required coverage.
- Insufficient staffing for a process on a particular day.

- Changes to an employee's schedule creating gaps across multiple brands.
- Configured scheduling rules or employee preferences being violated.

Distinguish between mandatory constraints and preferences. For example, a required client coverage day should take priority over a preferred weekly-off day.

Where a conflict cannot be resolved automatically, the application should clearly flag it for the operations manager.

E. Roster Views and Editing

Provide useful views for different stakeholders:

- **Company-wide view:** The complete monthly roster across all pods.
- **Pod-wise view:** Employees and schedules under each pod lead.
- **Brand-wise view:** Assigned employees, processes, and daily coverage.
- **Process-wise view:** Staffing and working schedules for each process.
- **Employee-wise view:** An individual's working days, weekly offs, holidays, and approved leaves.

The interface should make it easy to identify who is working on a given day and whether a brand's operational requirements are being met.

The operations manager should be able to edit assignments and schedules, with the system validating the changes.

F. Handling Changes

Roster planning is an ongoing process. Employees may request leave, approved leaves may change, and brands may introduce new operating requirements.

Demonstrate how your application handles at least one change after a roster has been generated.

For example, if an employee managing all processes for a brand becomes unavailable on a required working day, the system should flag the resulting coverage gap and help the operations manager identify a possible resolution.

Similarly, if one of three employees assigned to a brand is unavailable, the system should evaluate whether the remaining employees can maintain the required coverage.

4. Sample Data and Assumptions

You should create a realistic sample dataset containing at least 30 employees distributed across multiple brands, processes, and pods.

The dataset should demonstrate both single-resource brands and brands with multiple employees, including cases where employees handle separate processes and cases where multiple employees share a process.

You may create fictional brand names, employee names, pod assignments, operating calendars, weekly-off rules, public holidays, and coverage requirements.

Not all the information required to produce a valid roster will necessarily be provided. Make reasonable assumptions, document them clearly, and ensure that your application allows the relevant information to be configured.

Your solution should be flexible enough to support new brands, processes, employees, and working rules without requiring major changes to the codebase.

5. Technology and Architecture

The technology stack is entirely up to you. You may use any programming language, framework, database, hosting platform, or AI-assisted development tool you are comfortable with.

You may use tools such as Cursor, ChatGPT/Codex, GitHub Copilot, or other AI coding assistants.

Your solution may use deterministic scheduling logic, optimisation algorithms, AI-assisted workflows, or a combination of approaches.

We are not evaluating you on whether you write every line of code from scratch. We are interested in how you break down the problem, model the data, implement the scheduling logic, and use modern tools effectively.

You should be able to explain how your system works and why you chose your approach.

6. Optional: Go Beyond the Basics

The following features are optional:

- Fair distribution of weekly offs across employees.
- Preference-based scheduling.
- Automatic roster regeneration after approved leave or availability changes.
- Backup resource recommendations.
- Comparison of alternative roster plans.
- Roster approval workflows for pod leads and operations managers.
- Notifications when a published roster changes.
- Exporting rosters to CSV, Excel, or Google Sheets.
- **AI-assisted explanations of scheduling conflicts.**
- **Support for adding new brands, pods, and processes dynamically.**

Prioritise reliable scheduling, accurate coverage, and clear conflict detection over adding features for the sake of complexity.

7. Deployment and Submission

Your application should be publicly accessible so that it can be tested without setting up your development environment.

Please submit:

1. **Working application URL:** A deployed application with realistic sample data.
2. **GitHub repository:** Source code, README, setup instructions, and an `.env.example` file if applicable.
3. **Demo video:** A 3–5 minute walkthrough demonstrating roster generation, different brand schedules, conflict detection, and how the application handles a change.
4. **Approach note:** A short explanation of your solution, architecture, and key decisions.
5. **LinkedIn Profile:** Please include a link to your LinkedIn profile and make sure the link is accessible

8. Tell Us How You Think

Include a short section in your README answering the following questions:

1. How did you model employees, brands, processes, pods, and their relationships?
2. How does your system generate a roster and determine whether it satisfies the required constraints?
3. How did you handle the difference between single-resource brands and brands with multiple employees?
4. What happens when an employee manages all processes for a brand but is unavailable on a required working day?
5. How does your system handle conflicting requirements or insufficient staffing?
6. Which parts of the process did you automate, and which did you leave under human control? Why?
7. If the company doubled its team and brand count, what would you change about your solution?
8. If you had one additional week to work on this, what would you improve first?

There are no single correct answers. We want to understand your reasoning, implementation choices, and the trade-offs you made.

9. Submission

Please submit your completed assignment to:

- Primary: ozair.shaikh@datastraw.in | aryan.jaiswal@datastraw.in
- CC: talent@datastraw
- Subject: Smart Roster Management System Assignment - [Your Full Name]

Please include in your email:

1. Application URL
2. GitHub Repository
3. LinkedIn Profile Link (ensure working)
4. 2-3 sentence approach not

10. What We're Looking For

We are looking for someone who can take an open-ended operational problem, understand its constraints, use modern development tools effectively, build a working solution, and explain the decisions behind it.

A tool with reliable scheduling logic and clear conflict detection is more valuable than a visually impressive interface that generates invalid rosters.

Focus on solving the operational problem, not just building the interface.